



DulceByte

Proyecto final MOD 3.3, 3.4, 3.5 - Fase 2

Pruebas de la API REST

Estudiante

Melanie Hernández

Docente

Daniel Sosa

Emprendimiento

DulceByte - Cafetería y Repostería

Tecnologías

ASP.NET Core 8, Entity Framework Core, MySQL/MariaDB,
Swagger/OpenAPI

Fecha

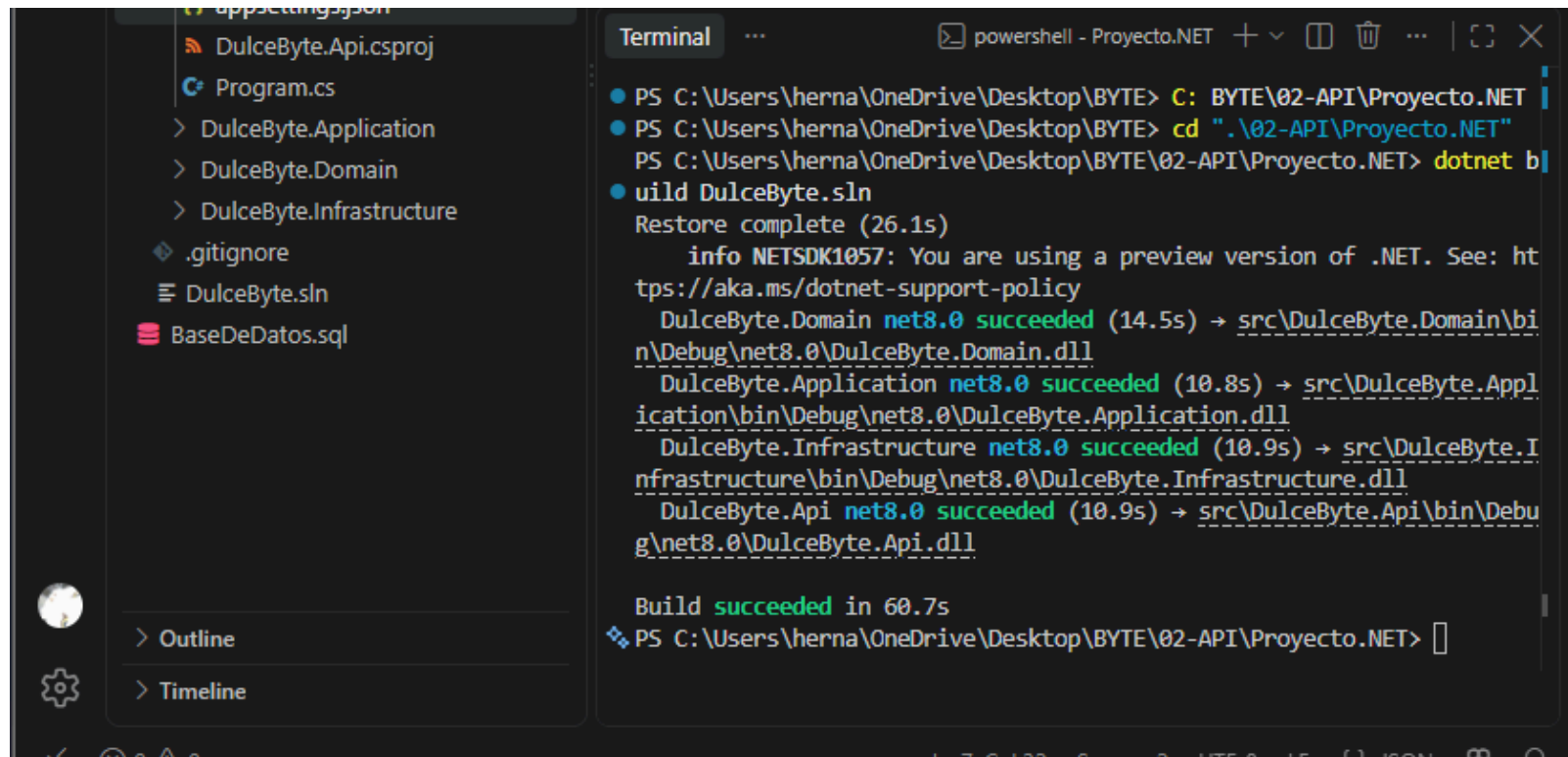
1 de septiembre de 2026

1. Objetivo de las pruebas

Comprobar que los endpoints de DulceByte funcionan sobre la base de datos real, utilizan métodos HTTP correctos, devuelven códigos de respuesta apropiados y permiten ejecutar el CRUD y la operación principal del negocio.

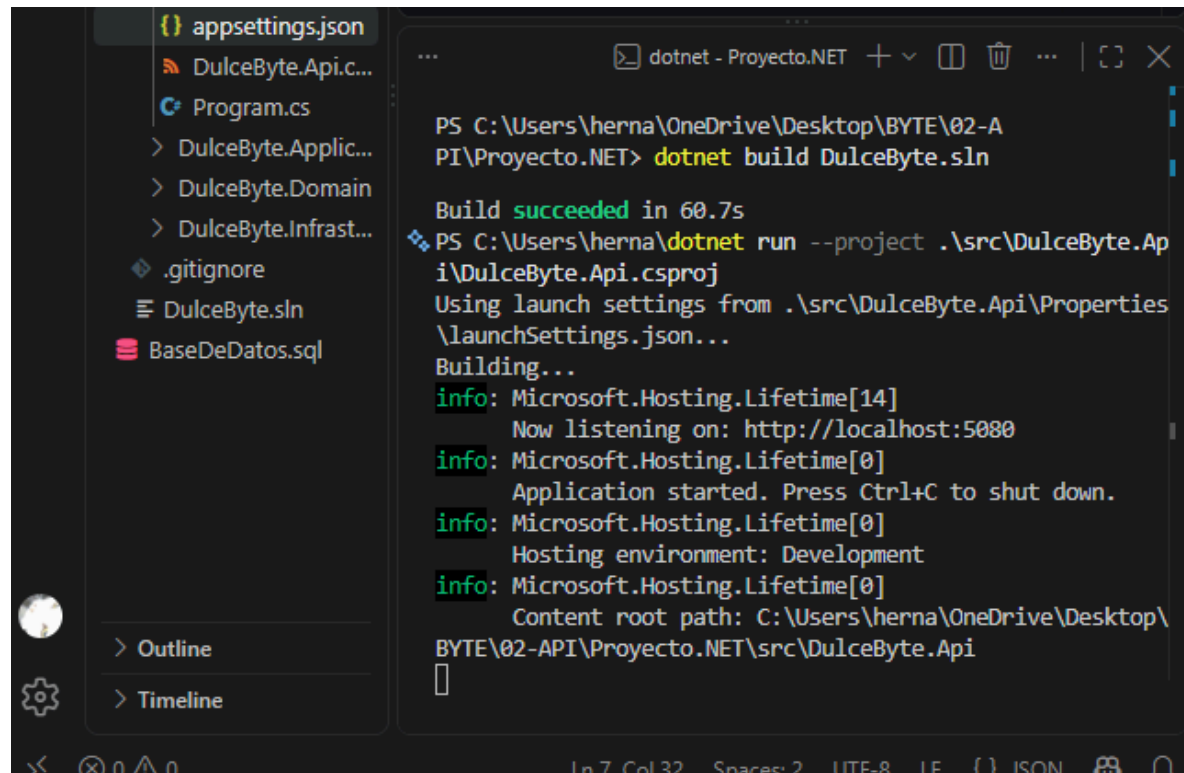
2. Entorno de prueba

- API: ASP.NET Core 8 ejecutada localmente en <http://localhost:5080>.
- Base de datos: dulcebytedb en MySQL/MariaDB mediante XAMPP.
- Herramienta: Swagger / OpenAPI.
- Compilación: solución DulceByte.sln compilada correctamente.



```
PS C:\Users\herna\OneDrive\Desktop\BYTE> C: BYTE\02-API\Proyecto.NET
PS C:\Users\herna\OneDrive\Desktop\BYTE> cd ".\02-API\Proyecto.NET"
PS C:\Users\herna\OneDrive\Desktop\BYTE\02-API\Proyecto.NET> dotnet build DulceByte.sln
Restore complete (26.1s)
info NETSDK1057: You are using a preview version of .NET. See: https://aka.ms/dotnet-support-policy
DulceByte.Domain net8.0 succeeded (14.5s) -> src\DulceByte.Domain\bin\Debug\net8.0\DulceByte.Domain.dll
DulceByte.Application net8.0 succeeded (10.8s) -> src\DulceByte.Application\bin\Debug\net8.0\DulceByte.Application.dll
DulceByte.Infrastructure net8.0 succeeded (10.9s) -> src\DulceByte.Infrastructure\bin\Debug\net8.0\DulceByte.Infrastructure.dll
DulceByte.Api net8.0 succeeded (10.9s) -> src\DulceByte.Api\bin\Debug\net8.0\DulceByte.Api.dll
Build succeeded in 60.7s
PS C:\Users\herna\OneDrive\Desktop\BYTE\02-API\Proyecto.NET>
```

Evidencia 1. Compilación completa de la solución: Build succeeded.



The screenshot shows the Visual Studio interface. On the left, the Solution Explorer displays the project structure for 'Proyecto.NET', including files like 'appsettings.json', 'DulceByte.Api.c...', 'Program.cs', and folders for 'DulceByte.Applic...', 'DulceByte.Domain', and 'DulceByte.Infrast...'. The main editor area shows the 'dotnet - Proyecto.NET' terminal window. The terminal output indicates a successful build and the application running on 'http://localhost:5080'. The output text is as follows:

```
PS C:\Users\herna\OneDrive\Desktop\BYTE\02-API\Proyecto.NET> dotnet build DulceByte.sln

Build succeeded in 60.7s
PS C:\Users\herna\OneDrive\Desktop\BYTE\02-API\Proyecto.NET> dotnet run --project .\src\DulceByte.Api\
DulceByte.Api.csproj
Using launch settings from .\src\DulceByte.Api\Properties\launchSettings.json...
Building...
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5080
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: C:\Users\herna\OneDrive\Desktop\BYTE\02-API\Proyecto.NET\src\DulceByte.Api
[]
```

Evidencia 2. API ejecutándose en <http://localhost:5080>.

3. Pruebas de consulta (GET)

3.1 GET /api/categorias

Endpoint	/api/categorias
Método HTTP	GET
Parámetros	Sin parámetros
Respuesta	Lista de categorías Bebidas y Repostería
Código HTTP	200 OK

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  http://localhost:5000/api/categorias \
  -H 'accept: text/plain'
```

Request URL

http://localhost:5000/api/categorias

Server response

Code

Details

200

Response body

```
{
  "idCategoria": 0,
  "nombre": "Bebidas",
  "descripcion": "Cafés y bebidas disponibles en DulceByte"
},
{
  "idCategoria": 0,
  "nombre": "Repostería",
  "descripcion": "Pastelas, brownies, galletas y postres"
}

```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Tue, 01 Sep 2026 02:22:25 GMT
server: Kestrel
transfer-encoding: chunked
```

Responses

Code

Description

Links

200

OK

No links

Media type

text/plain

Controls Accept header.

Example Value | Schema

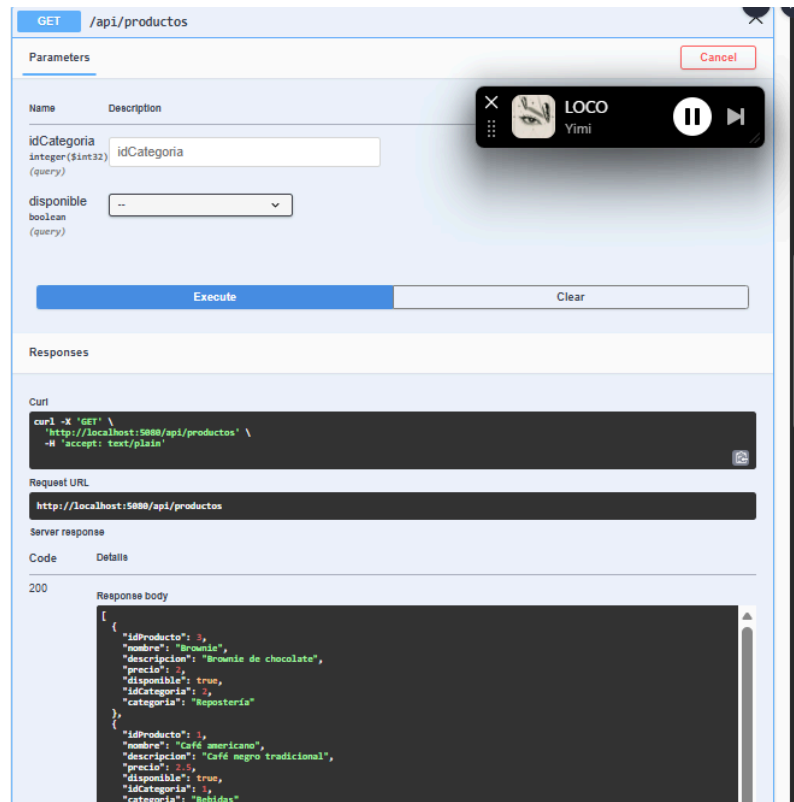
```
{
  "idCategoria": 0,
  "nombre": "string",
  "descripcion": "string"
}

```

Evidencia GET /api/categorias: respuesta 200 OK.

3.2 GET /api/productos

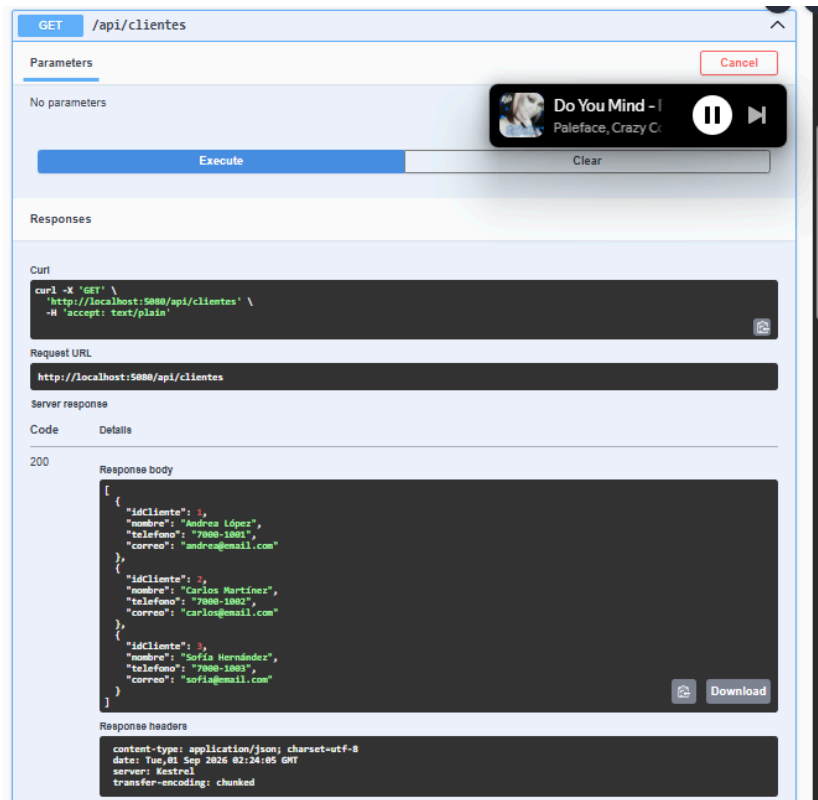
Endpoint	/api/productos
Método HTTP	GET
Parámetros	Filtros opcionales: idCategoria, disponible
Respuesta	Lista de productos con categoría
Código HTTP	200 OK



Evidencia GET /api/productos: respuesta 200 OK.

3.3 GET /api/clientes

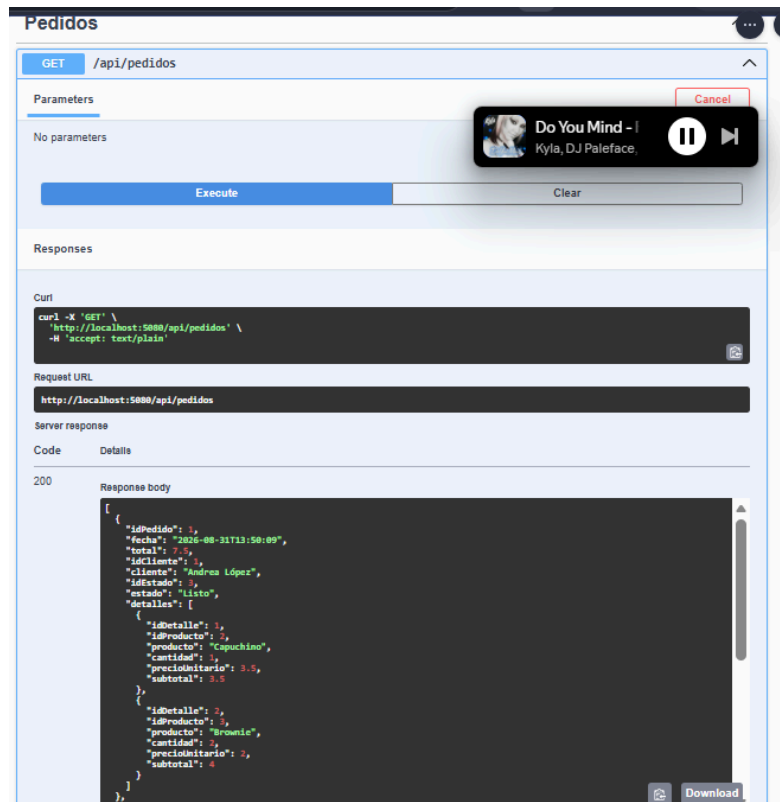
Endpoint	/api/clientes
Método HTTP	GET
Parámetros	Sin parámetros
Respuesta	Lista de clientes registrados
Código HTTP	200 OK



Evidencia GET /api/clientes: respuesta 200 OK.

3.4 GET /api/pedidos

Endpoint	/api/pedidos
Método HTTP	GET
Parámetros	Sin parámetros
Respuesta	Pedidos con cliente, estado y detalles
Código HTTP	200 OK



Evidencia GET /api/pedidos: respuesta 200 OK.

4. CRUD completo de Productos

4.1 CREATE - POST /api/productos

Endpoint	/api/productos
Método HTTP	POST
Parámetros	Sin parámetros de ruta
Body enviado	<pre>{ "nombre": "Muffin de vainilla", "descripcion": "Muffin artesanal de vainilla", "precio": 2.75, "disponible": true, "idCategoria": 2 }</pre>
Respuesta	Producto creado con idProducto = 5 y categoría Repostería

Responses

Curli

```
curl -X 'POST' \
  'http://localhost:5000/api/productos' \
  -H 'accept: text/plain' \
  -H 'Content-Type: application/json' \
  -d '{
    "nombre": "Muffin de vainilla",
    "descripcion": "Muffin artesanal de vainilla",
    "precio": 2.75,
    "disponible": true,
    "idCategoria": 2
  }'
```

Request URL

http://localhost:5000/api/productos

Server response

Code	Details
201 Undocumented	Response body <pre>{ "idProducto": 1, "nombre": "Muffin de vainilla", "descripcion": "Muffin artesanal de vainilla", "precio": 2.75, "disponible": true, "idCategoria": 2, "categoria": "Repostería" }</pre> Response headers <pre>access-control-allow-origin: * content-type: application/json; charset=utf-8 date: Tue, 01 Sep 2026 02:28:22 GMT location: http://localhost:5000/api/productos/5 server: Kestrel transfer-encoding: chunked</pre>

Responses

Code	Description	Links
200	OK	No links

Media type

text/plain

Controls Accept header.

Evidencia CREATE. Swagger devuelve 201 y el recurso creado.

4.2 READ - GET /api/productos/5

The screenshot shows a REST client interface with the following sections:

- Method and URL:** GET /api/productos/{id}
- Parameters:** A table with columns 'Name' and 'Description'. It contains one parameter: 'id' (required, integer(\$int32), path) with a value of '5'. There is a 'Cancel' button.
- Buttons:** 'Execute' and 'Clear' buttons.
- Responses:**
 - Curl:** A code block showing the curl command:

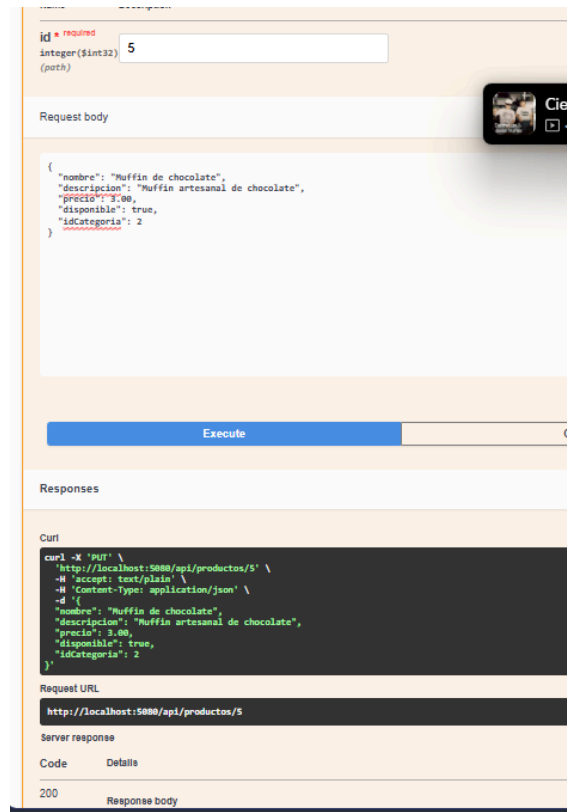
```
curl -X 'GET' \
'http://localhost:5000/api/productos/5' \
-H 'accept: text/plain'
```
 - Request URL:** A text box containing 'http://localhost:5000/api/productos/5'.
 - Server response:** A table with columns 'Code' and 'Details'. It shows a response with code '200' and a 'Response body' field.

Evidencia READ. Consulta del producto 5 con respuesta 200 OK.

4.3 UPDATE - PUT /api/productos/5

Body enviado:

```
{
  "nombre": "Muffin de chocolate",
  "descripcion": "Muffin artesanal de chocolate",
  "precio": 3.00,
  "disponible": true,
  "idCategoria": 2
}
```



Evidencia UPDATE. El producto 5 se modifica y la API responde 200 OK.

4.4 DELETE - DELETE /api/productos/5

DELETE

/api/productos/{id}

Parameters

Name	Description
id * required	
integer(\$int32)	5
(path)	

Execute

Clear

Responses

Curl

```
curl -X 'DELETE' \
'http://localhost:5080/api/productos/5' \
-H 'accept: */*'
```

Request URL

```
http://localhost:5080/api/productos/5
```

Server response

Code	Details
204	Response headers <pre>access-control-allow-origin: * date: Tue, 01 Sep 2026 02:35:09 GMT server: Kestrel</pre>

Responses

Code	Description
200	OK

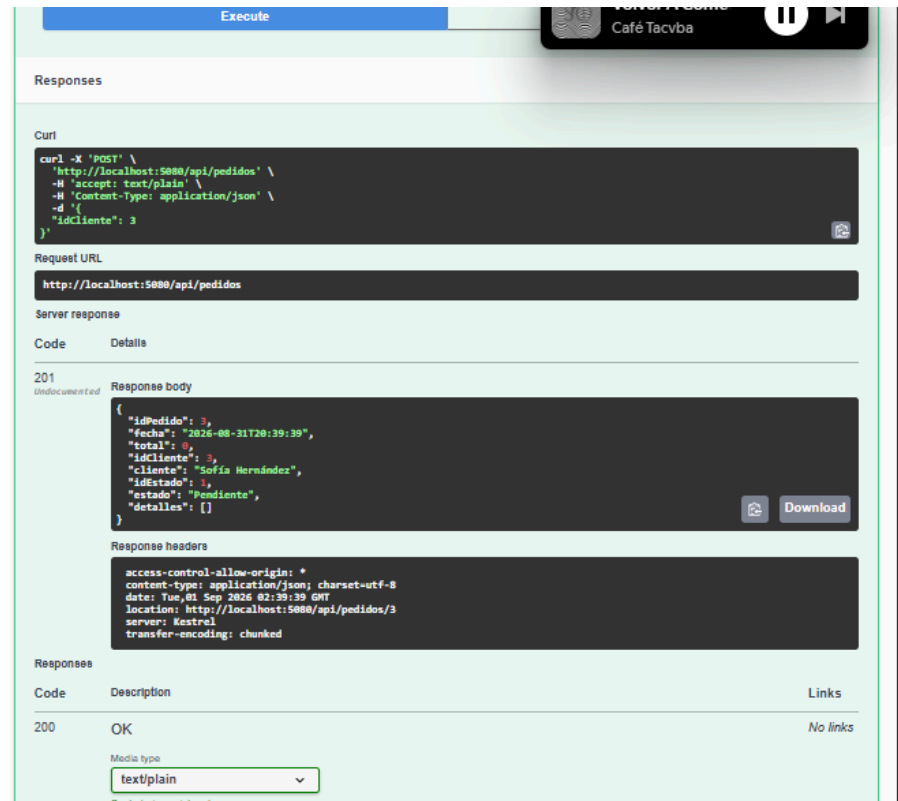
Evidencia DELETE. Eliminación exitosa con código 204 No Content.

5. Operación principal del negocio: gestionar un pedido

Se probó el flujo crear pedido -> agregar producto -> recalculr total -> actualizar estado. El pedido de prueba pertenece a Sofía Hernández (idCliente = 3).

5.1 Crear pedido

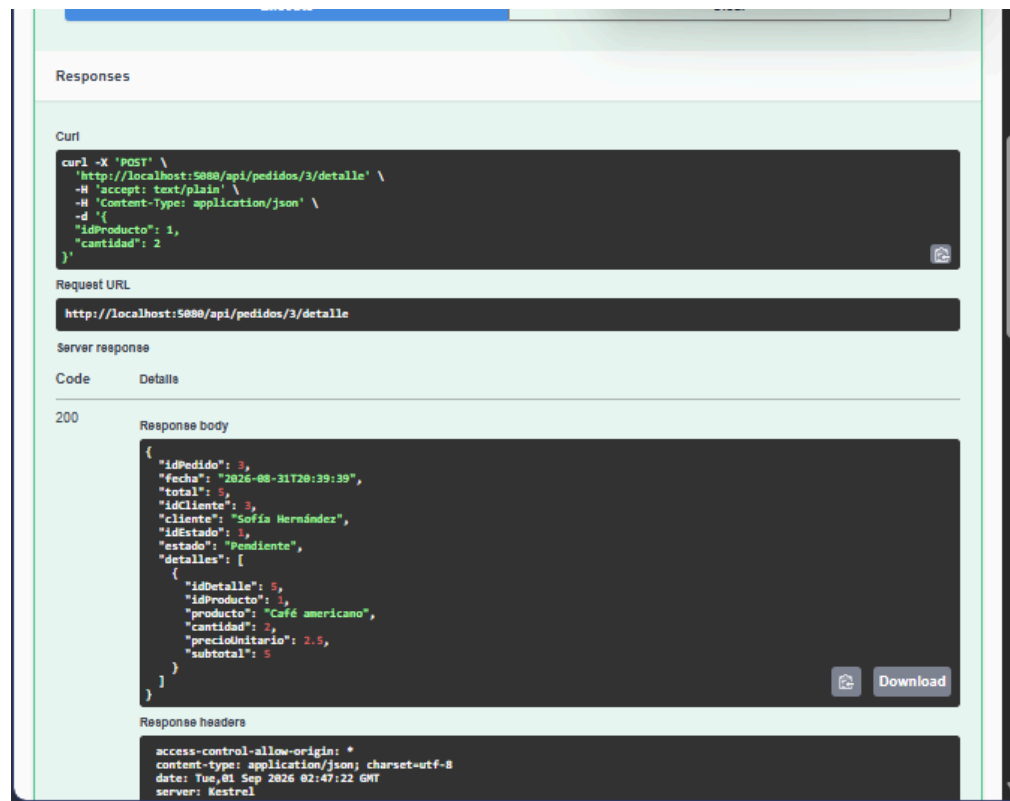
Endpoint	/api/pedidos
Método HTTP	POST
Parámetros	id de pedido cuando aplica
Body enviado	{ "idCliente": 3 }
Respuesta	Pedido #3 creado, estado Pendiente y total 0
Código HTTP	201 Created



Evidencia 5.1 Crear pedido: 201 Created.

5.2 Agregar detalle

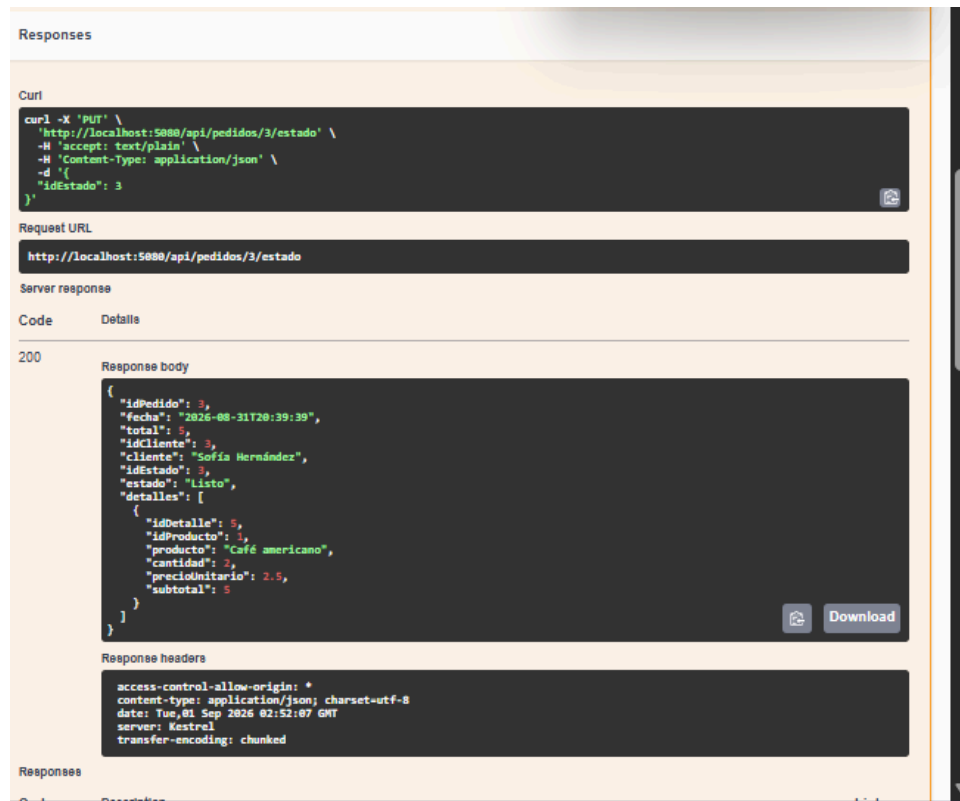
Endpoint	/api/pedidos/3/detalle
Método HTTP	POST
Parámetros	id de pedido cuando aplica
Body enviado	{ "idProducto": 1, "cantidad": 2 }
Respuesta	Café americano x2; subtotal y total = 5.00
Código HTTP	200 OK



Evidencia 5.2 Agregar detalle: 200 OK.

5.3 Actualizar estado

Endpoint	/api/pedidos/3/estado
Método HTTP	PUT
Parámetros	id de pedido cuando aplica
Body enviado	{ "idEstado": 3 }
Respuesta	Pedido #3 cambia a estado Listo y conserva total = 5.00
Código HTTP	200 OK



Evidencia 5.3 Actualizar estado: 200 OK.

6. Resumen de resultados

Prueba	Endpoint	HTTP	Resultado
Consulta de categorías	GET /api/categorias	200	Aprobada
Consulta de productos	GET /api/productos	200	Aprobada
Consulta de clientes	GET /api/clientes	200	Aprobada
Consulta de pedidos	GET /api/pedidos	200	Aprobada
Crear producto	POST /api/productos	201	Aprobada
Consultar producto por ID	GET /api/productos/5	200	Aprobada
Actualizar producto	PUT /api/productos/5	200	Aprobada
Eliminar producto	DELETE /api/productos/5	204	Aprobada
Crear pedido	POST /api/pedidos	201	Aprobada
Agregar producto al pedido	POST /api/pedidos/3/detalle	200	Aprobada
Actualizar estado del pedido	PUT /api/pedidos/3/estado	200	Aprobada

7. Observación final

Las pruebas demuestran que la API se comunica con la base de datos real y que el CRUD y el flujo principal de pedidos funcionan correctamente. Como verificación adicional antes de la entrega puede ejecutarse una consulta a un identificador inexistente (por ejemplo, GET /api/productos/9999) para capturar explícitamente una respuesta 404 del middleware de errores; esta prueba es complementaria y no modifica los resultados funcionales ya obtenidos.